Experiment 5

Name : Astik Joshi **UID :** 23BCS10627
Branch: BE-CSE **Section/Group:** KRG-3B
Semester: 6th **Date of Performance:** -17/02/2026
Subject Name: Full Stack-II **Subject Code:** 23CSH-309

Aim: The aim of this implementation is to verify the correctness and reliability of the EcoTrack React application by writing automated tests using Jest and React Testing Library, and by analyzing application behavior using debugging tools.

Objective:-

- Understand the role and importance of automated testing in frontend applications and write unit tests for JavaScript utility functions using Jest with appropriate matchers.
- Test React components using React Testing Library, validate UI rendering by querying DOM elements, and ensure expected component behavior.
- Implement asynchronous testing using methods like findBy and waitFor, and apply mocking techniques to simulate API calls or external data responses.
- Perform snapshot testing to identify unintended UI changes and maintain consistency in component output.
- Debug failing tests and application logic using browser Developer Tools and breakpoints, and analyze application behavior systematically instead of relying on manual testing.

Implementation/Code:

The **EcoTrack application** uses **Redux Toolkit** for centralized state management, ensuring predictable data flow and easier debugging. A global Redux store is configured using configureStore with feature-based slices like logsSlice, while asynchronous operations are handled using **Redux Async Thunks** to manage API request states. **React-Redux hooks** (useDispatch, useSelector) connect components to the store, enabling data fetching and state access. **Selectors** derive filtered views from the Redux state without mutation, and loading and error states are managed to improve performance and user experience.

- **/store/logSlice.js :-**

```
import { createSlice,createAsyncThunk } from "@reduxjs/toolkit";
export const fetchLogs = createAsyncThunk(
    'logs/fetchLogs',
    async () => {
        await new Promise(resolve => setTimeout(resolve, 1000));
        return [
            { id: 1,activity:"Car Travel",carbon:4},
            { id: 2,activity:"Electricity Usage",carbon:6},
            { id: 3,activity:"Cycling",carbon:0},
        ];
    }
);

const logsSlice = createSlice({
    name: 'logs',
    initialState: {
        data: [],
        status: 'idle',
        error: null,
    },
    reducers: {},
    extraReducers: (builder) => {
        builder
            .addCase(fetchLogs.pending, (state) => {
                state.status = 'loading';
            })
            .addCase(fetchLogs.fulfilled, (state, action) => {
                state.status = 'succeeded';
                state.data = action.payload;
            })
            .addCase(fetchLogs.rejected, (state, action) => {
                state.status = 'failed';
                state.error = action.error.message;
            });
    },
});
export default logsSlice.reducer;
```

ReportVitals.js

src > JS reportWebVitals.js > ...

```javascript
const reportWebVitals = onPerfEntry => {
  if (onPerfEntry && onPerfEntry instanceof Function) {
    import('web-vitals').then(({ getCLS, getFID, getFCP, getLCP, getTTFB }) => {
      getCLS(onPerfEntry);
      getFID(onPerfEntry);
      getFCP(onPerfEntry);
      getLCP(onPerfEntry);
      getTTFB(onPerfEntry);
    });
  }
};

export default reportWebVitals;
```

Setup.js

src > JS setupTests.js

```javascript
// jest-dom adds custom jest matchers for asserting on DOM nodes.
// allows you to do things like:
// expect(element).toHaveTextContent(/react/i)
// learn more: https://github.com/testing-library/jest-dom
import '@testing-library/jest-dom';
```

store.js :-

```javascript
import {configureStore} from '@reduxjs/toolkit';
import logsReducer from './logsSlice';
const store = configureStore({
    reducer: {
        logs: logsReducer,
    },
});
export default store;
```

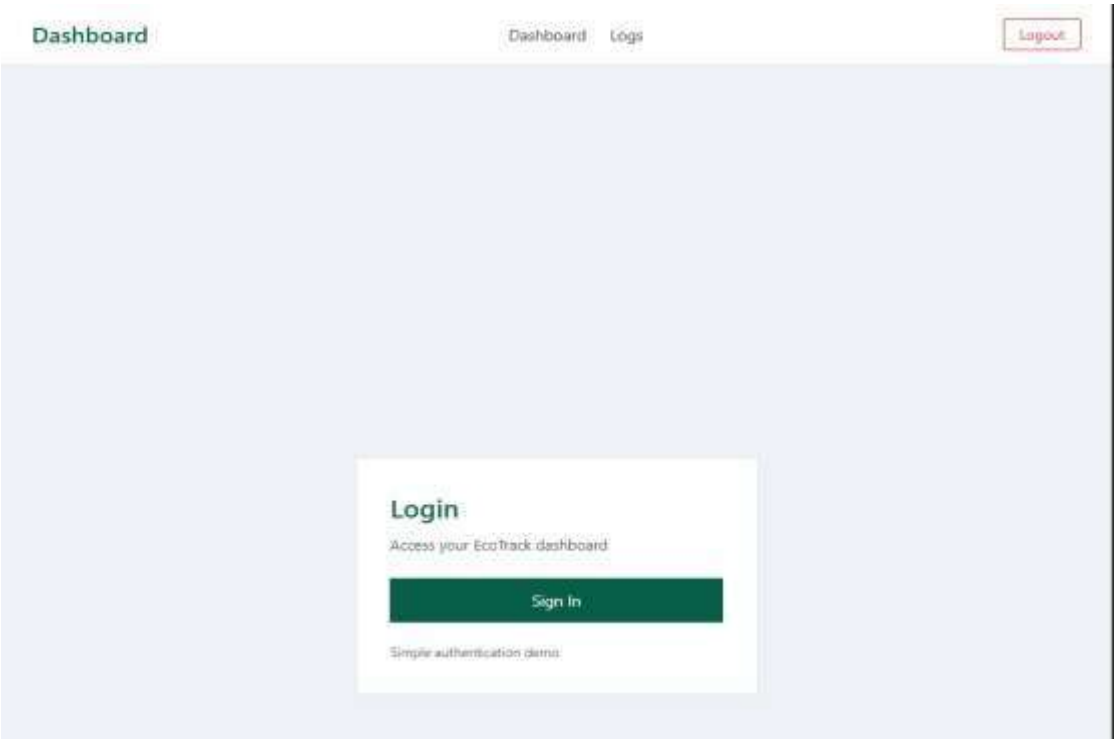
- **App.jsx:-**

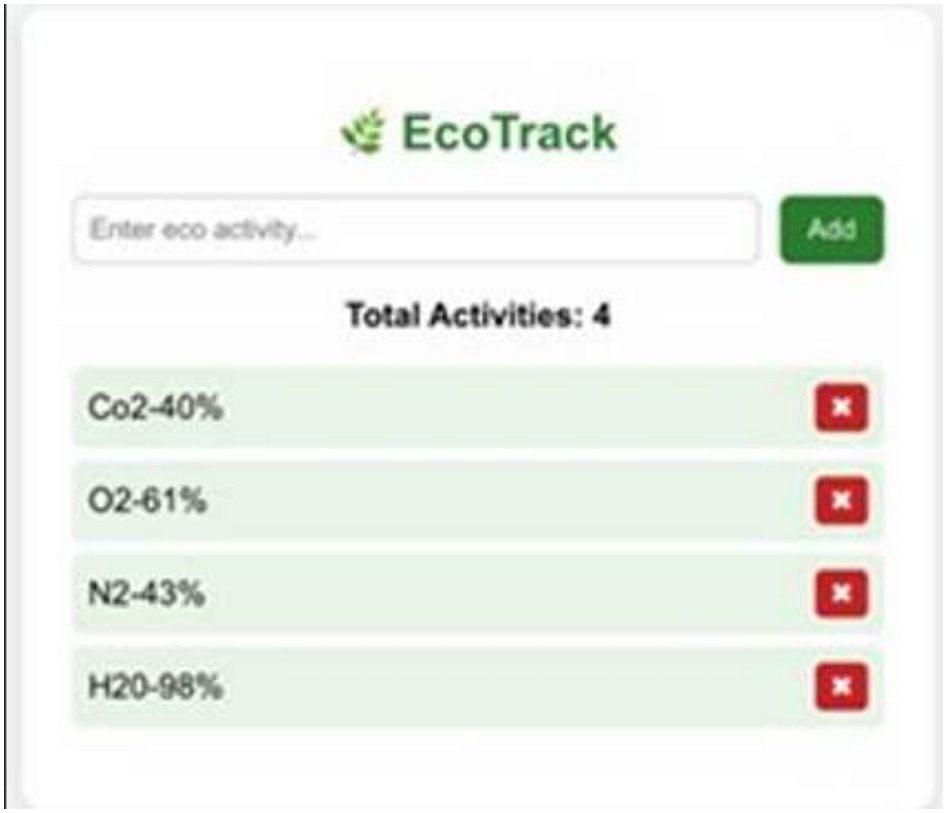
```jsx
import Dashboard from "./pages/dashboard.jsx";
import Log from "./pages/log.jsx";
import Header from "./components/Header.jsx";
import { Routes, Route, Link, BrowserRouter } from "react-router-dom";
import Login from "./pages/login.jsx";
import ProtectedRoute from "./Routes/ProtectedRoute.jsx";
import DashboardLayout from "./pages/dashboardLayout.jsx";

function App() {
  return (
    <>
      <BrowserRouter>
        <Header title="Dashboard" />
        <main>
          <Routes>
            <Route path="/login" element={<Login />} />
            <Route
              path="/dashboardLayout"
              element={
                <ProtectedRoute>
                  <DashboardLayout />
                </ProtectedRoute>
              }
            />
            <Route
              path="/"
              element={
                <ProtectedRoute>
                  <Dashboard />
                </ProtectedRoute>
              }
            />
            <Route
              path="/log"
              element={
                <ProtectedRoute>
                  <Log />
                </ProtectedRoute>
              }
            />
          </Routes>
        </main>
      </BrowserRouter>
    </>
  );
}

export default App;
```

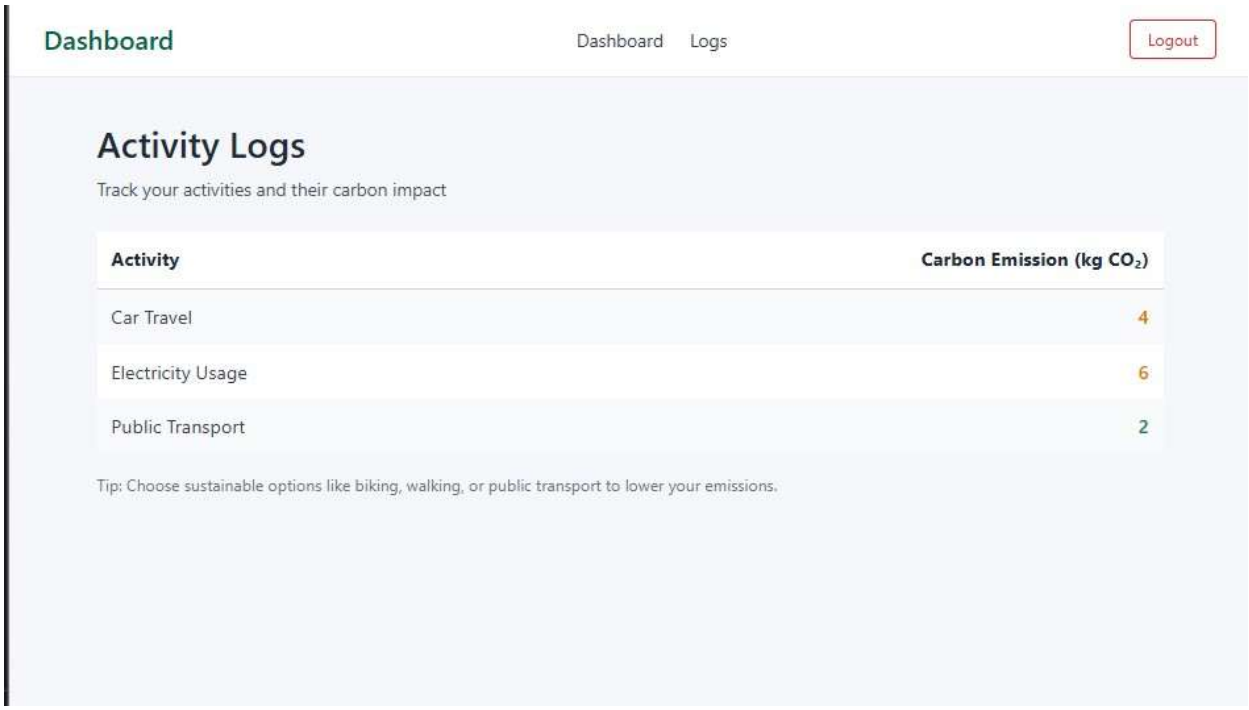
- ProtectRoute.jsx

```jsx
import {Navigate} from "react-router-dom";
import {useAuth} from "../context/AuthContext";

const ProtectedRoute = ({children}) => {
    const {isAuthenticated} = useAuth();

    if(!isAuthenticated){
        return <Navigate to="/login" replace />;
    }
    return children;
}
export default ProtectedRoute;
```

4. Output




EcoTrack
Enter eco activity...
Add
Total Activities: 4
Co2-40%
O2-61%
N2-43%
H20-98%


Dashboard
Dashboard
Logs
Logout
Activity Logs
Track your activities and their carbon impact
Activity
Carbon Emission (kg CO_2)
Car Travel
4
Electricity Usage
6
Public Transport
2
Tip: Choose sustainable options like biking, walking, or public transport to lower your emissions.

5. Learning Outcome :-

- Importance of automated testing in frontend applications
- Writing unit tests using Jest framework
- Using matchers like toBe() and toMatchSnapshot()
- Testing React components with React Testing Library
- Validating UI rendering through DOM queries
- Understanding snapshot testing for UI stability